

**Министерство науки и высшего образования Российской Федерации**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ**  
**УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**  
**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Факультет безопасности информационных технологий**

**Дисциплина:**  
**«Операционные системы»**

**ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2**  
**«Membomb»**

**Выполнил:**  
Пахомов Владимир Юрьевич, студент группы N3250



---

(подпись)

**Проверил:**  
Ханов Артур Рафаэлевич, преподаватель ФБИТ

---

(отметка о выполнении)

---

(подпись)

Санкт-Петербург  
2025г.

## Содержание

|                                       |   |
|---------------------------------------|---|
| Реализация Membomb на Arch Linux..... | 3 |
| Реализация Membomb на Windows 10..... | 5 |

## Реализация Membomb на Arch Linux

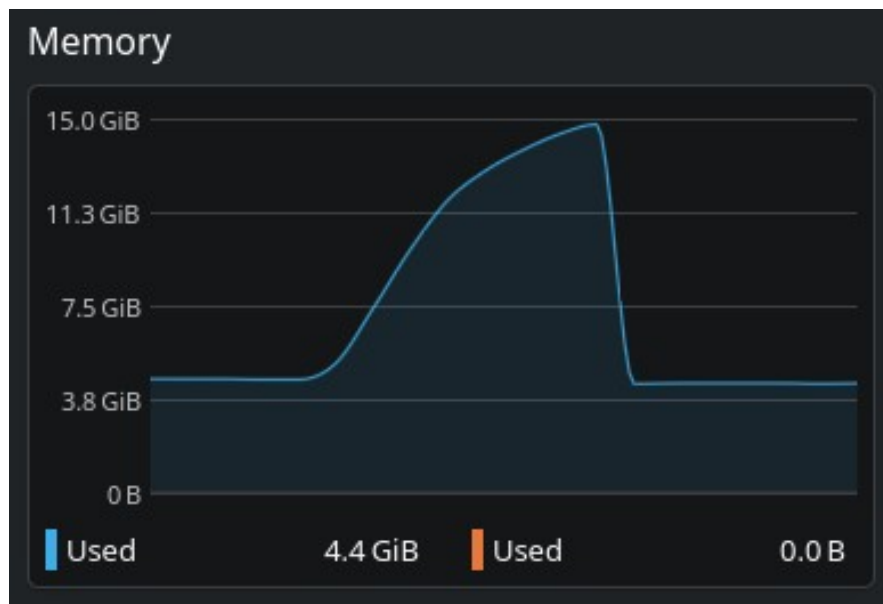


Рисунок 1: График потребления памяти за 10 секунд (KDE System Monitor)

Вывод команды `sudo dmesg | grep -i "oom"` (работа OOM Killer'a):

```
x vladimir@vladimirp ~$ sudo dmesg | grep -i "oom"
[23458.182868] gmain invoked oom-killer: gfp_mask=0x140cca(GFP_HIGHUSER_MOVABLE|__GFP_COMP), order=0, oom_score_adj=0
[23458.182906] oom_kill_process.cold+0x8/0x90
[23458.183234] [ pid ] uid tgid total_vm rss rss_anon rss_file rss_shmem pgtables_bytes swapents oom_score_adj name
[23458.184038] oom-kill:constraint=CONSTRAINT_NONE,nodemask=(null),cpuset=/,mems_allowed=0,global_oom,task_memcg=/user.slice/user-1000.slice
@1000.service/app.slice/app-org.kde.konsole@ade3da26253942ebbaf59f26dbf8d05f.service,task=membomb,pid=23817,uid=1000
[23458.184058] Out of memory: Killed process 23817 (membomb) total-vm:15731348kB, anon-rss:11078820kB, file-rss:252kB, shmem-rss:0kB, UID:10
tables:21728kB oom_score_adj:200
vladimir@vladimirp ~$
```

Рисунок 2: Вывод работы OOM Killer Linux

Исходный код `membomb.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

#define MB_T0_ALLOC 1024*15

int main() {
    long page_size = sysconf(_SC_PAGESIZE);

    size_t total_bytes_to_alloc = (size_t)MB_T0_ALLOC * 1024 * 1024;
    char *mem_ptr = NULL;
    size_t allocated_so_far = 0;
    size_t offset = 0;
```

```

    mem_ptr = (char *)mmap(NULL, total_bytes_to_alloc, PROT_READ |
PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

    printf("Start allocating memory...\n");

    for (offset = 0; offset < total_bytes_to_alloc; offset += page_size) {
        mem_ptr[offset] = 0;
        allocated_so_far += page_size;
        if (allocated_so_far % (100 * 1024 * 1024) == 0) {
            printf("Allocated: %.2f MB\n", (double)allocated_so_far / (1024 *
1024));
        }
    }

    printf("Total allocated memory: %.2f MB\nPress Enter to free\n",
(double)allocated_so_far / (1024 * 1024));
    getchar();

    return 0;
}

```

## Реализация Membomb на Windows 10

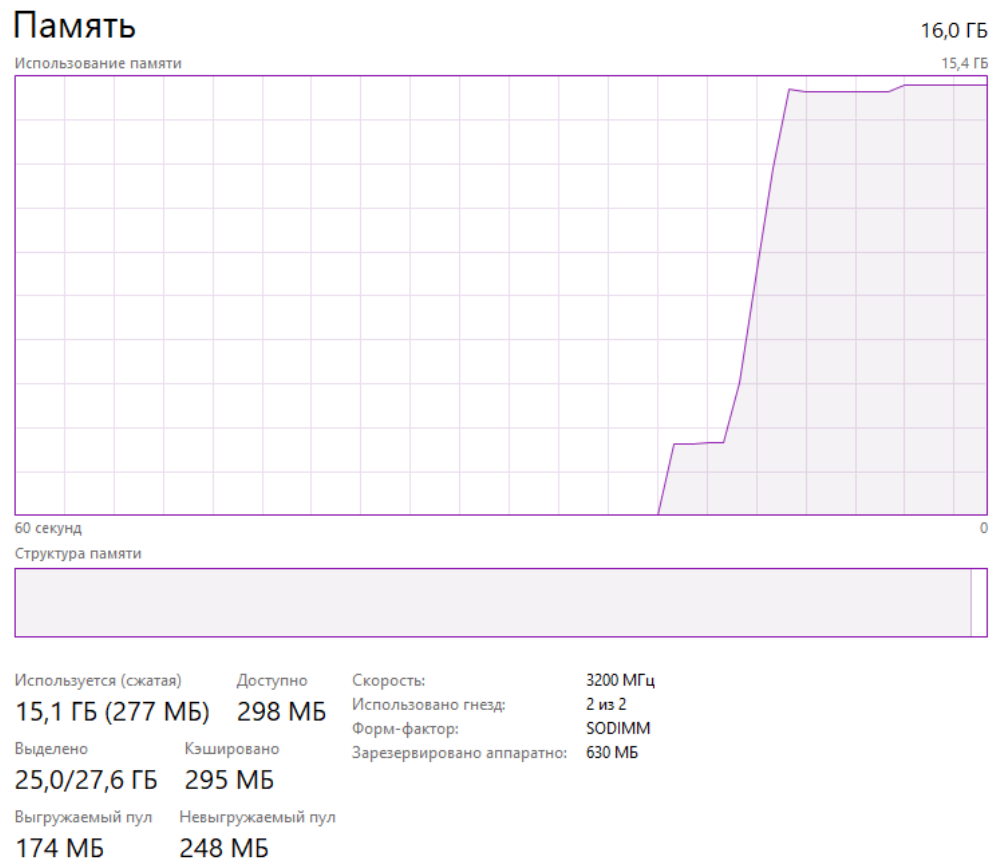


Рисунок 3: График потребления памяти (Системный монитор Windows)

Ошибка выделения памяти:

```
Allocated 23132 MB...
Allocated 23136 MB...
Allocated 23140 MB...
Allocated 23144 MB...
Allocated 23148 MB...
Allocated 23152 MB...
Allocated 23156 MB...
Allocated 23160 MB...
Allocated 23164 MB...
Allocated 23168 MB...
Can't allocate memory. Error code: 1455
Total allocated memory: 23168 MB
Press Enter to free
```

Рисунок 4: Ошибка выделения памяти в Windows

## Исходный код membomb\_win.c:

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ALLOC_STEP_BYTES (1024 * 1024 * 4)
#define MAX_BLOCKS 100000
void* allocated_memory_blocks[MAX_BLOCKS];
int num_allocated_blocks = 0;

int main() {
    long long total_allocated_bytes = 0;
    DWORD error_code;

    printf("Start allocating memory...\n");

    while (num_allocated_blocks < MAX_BLOCKS) {
        void* ptr = VirtualAlloc(
            NULL,
            ALLOC_STEP_BYTES,
            MEM_COMMIT | MEM_RESERVE,
            PAGE_READWRITE
        );

        if (ptr == NULL) {
            error_code = GetLastError();
            fprintf(stderr, "Can't allocate memory. Error code: %lu\n",
error_code);
            break;
        }

        memset(ptr, 0, ALLOC_STEP_BYTES);

        allocated_memory_blocks[num_allocated_blocks++] = ptr;
        total_allocated_bytes += ALLOC_STEP_BYTES;

        printf("Allocated %lld MB...\n", total_allocated_bytes / (1024 *
1024));
    }

    printf("Total allocated memory: %lld MB\nPress Enter to free\n",
total_allocated_bytes / (1024 * 1024));

    getchar();

    printf("Free memory...\n");
    for (int i = 0; i < num_allocated_blocks; ++i) {
        VirtualFree(allocated_memory_blocks[i], 0, MEM_RELEASE);
    }

    return 0;
}
```